

COMPUTER NETWORKS

UNIT – IV

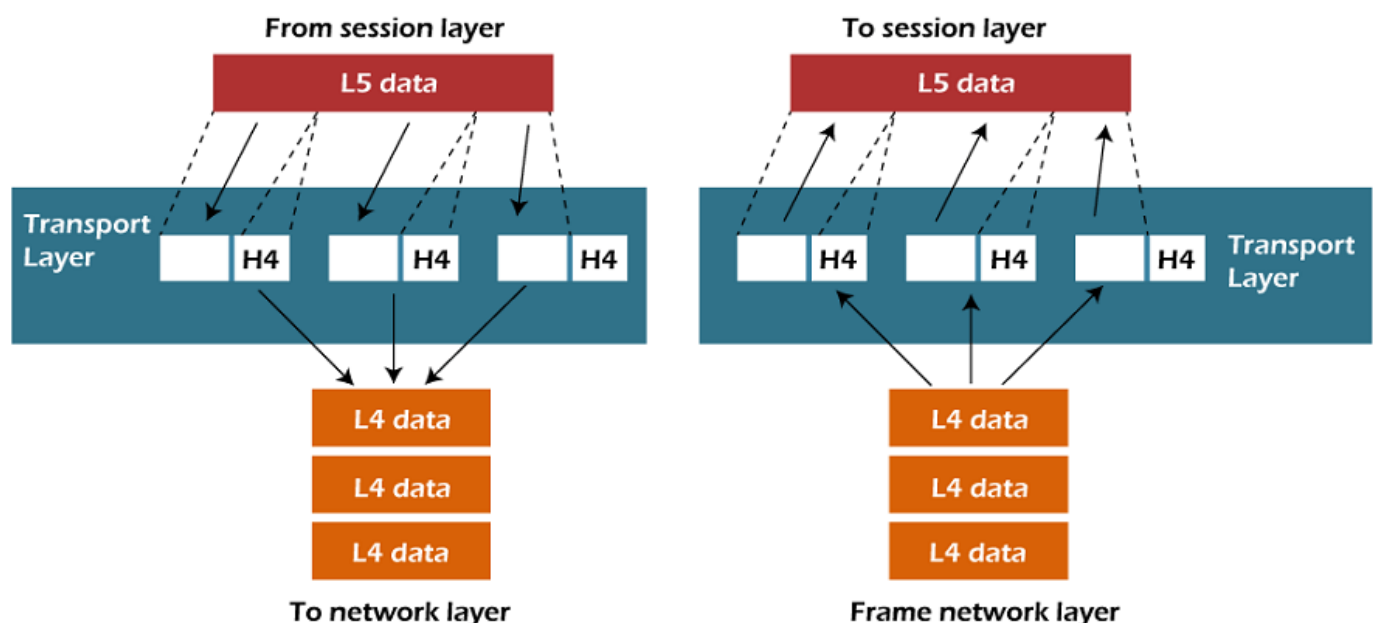
TRANSPORT LAYER

Layer 4 of the OSI model, also known as the transport layer, manages network traffic between hosts and end systems to ensure complete data transfers. Transport-layer protocols such as TCP, UDP, DCCP, and SCTP are used to control the volume of data, where it is sent, and at what rate.

By providing standardized access to communication services such as connection-oriented communication, reliability, flow control, and multiplexing, layer 4 eliminates the need for the application-oriented layers 5 – 7 to consider the characteristics of the communications network itself. Layer 4 is also responsible for end-to-end error recovery.

Because layer 4 coordinates data transfer without visibility into message content, layer 4 load balancing can make routing decisions without the need to decrypt or inspect network traffic. This makes layer 4 load balancing a quick and efficient approach for packet-level load balancing based on simple algorithms such as round-robin routing.

To route traffic based on media type, localization rules, application, or other criteria where message inspection is required, layer 7 load balancing must be used.



The two protocols used in this layer are:

- **Transmission Control Protocol**
 - It is a standard protocol that allows the systems to communicate over the internet.
 - It establishes and maintains a connection between hosts.
 - When data is sent over the TCP connection, then the TCP protocol divides the data into smaller units known as segments. Each segment travels over the internet using multiple routes, and they arrive in different orders at the destination. The transmission control protocol reorders the packets in the correct order at the receiving end.
- **User Datagram Protocol**
 - User Datagram Protocol is a transport layer protocol.
 - It is an unreliable transport protocol as in this case receiver does not send any acknowledgment when the packet is received, the sender does not wait for any acknowledgment. Therefore, this makes a protocol unreliable.

Functions of Transport Layer:

- **Service-point addressing:** Computers run several programs simultaneously due to this reason, the transmission of data from source to the destination not only from one computer to another computer but also from one process to another process. The transport layer adds the header that contains the address known as a service-point address or port address. The responsibility of the network layer is to transmit the data from one computer to another computer and the responsibility of the transport layer is to transmit the message to the correct process.
- **Segmentation and reassembly:** When the transport layer receives the message from the upper layer, it divides the message into multiple segments, and each segment is assigned with a sequence number that uniquely identifies each segment. When the message has arrived at the destination, then the transport layer reassembles the message based on their sequence numbers.
- **Connection control:** Transport layer provides two services Connection-oriented service and connectionless service. A connectionless service treats each segment as an individual packet, and they all travel in different routes to reach the destination. A connection-oriented service makes a connection with the transport layer at the destination machine before delivering the packets. In connection-oriented service, all the packets travel in the single route.
- **Flow control:** The transport layer also responsible for flow control but it is performed end-to-end rather than across a single link.
- **Error control:** The transport layer is also responsible for Error control. Error control is performed end-to-end rather than across the single link. The sender transport layer ensures that message reach at the destination without any error.

1. Reliable and Unreliable Connections

The transport layer uses reliable and unreliable connections to transfer data between two devices in a network. A reliable connection provides guaranteed data delivery, but it takes more time for data delivery than an unreliable connection. An unreliable connection delivers data faster than a reliable connection, but it does not provide any guarantee for data delivery.

Reliable connections

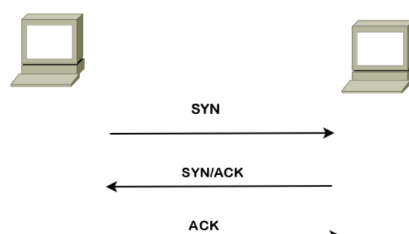
For reliable connections, it uses TCP (Transmission Control Protocol). The TCP protocol uses a three-way handshake, windowing, and sequence numbers for guaranteed data delivery.

Let us take an example to understand the meaning of each technique and how TCP uses it for guaranteed data delivery.

A user on PC-1 wants to transfer some files to a user on PC-2. Since these files require guaranteed data delivery, the transport layer on PC-1 implements the TCP protocol and delegates this responsibility.

The three-way handshake process

Before transmitting the data to PC-2, TCP ensures that PC-2 is up and ready to receive data. For this, it uses the three-way handshake process. The following steps are involved in this process.



The source (PC-1) sends a SYN (synchronization) segment to the destination (PC-2). This segment indicates PC-1 wants to establish a reliable session for data transfer.

PC-2 responds with a segment that includes acknowledgment and synchronization. The acknowledgment indicates the successful receipt of the source's SYN segment. The SYN flag indicates the destination is ready to establish the connection.

Upon receiving the SYN/ACK segment, PC-1 responds with an ACK segment. This indicates to PC-2 that its SYN was received by PC-1 and that the session is now fully established.

After going through the three-way handshake process, both PCs can transfer data across the session. TCP goes through this process whenever it transfers data. In this process, it also negotiates parameters it will use in data transmission.

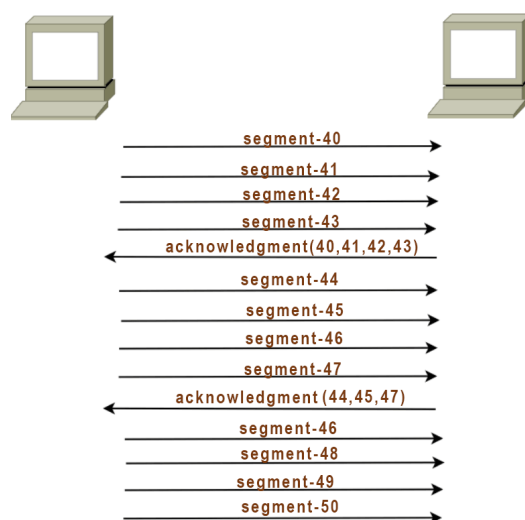
Windowing

Windowing is the number of segments the source can send before getting an acknowledgment from the destination. For example, if the window size is 4, PC-1 can send four segments before getting an acknowledgment from PC-2. After that, it must wait for an acknowledgment from PC-2. It will send the next four segments only when it will receive an acknowledgment from PC-2 for the sent segments.

Sequencing

To track each segment, TCP attaches an incremental number to each segment. The destination device uses these numbers to tell the source device which segments it received and which segments are lost in the transmission.

Let's take an example.



Suppose the window size is 4. PC-1 sends four segments whose sequence numbers are 40, 41, 42, and 43.

PC-2 receives all segments and send an acknowledgment indicating it received segment number 40, 41, 42, and 43 and is ready for the next four segments.

PC-1 sends the next four segments whose sequence numbers are 44, 45, 46, 47. Let's suppose this time segment 46 is lost in the middle.

PC-2 sends an acknowledgment indicating it received segment numbers 44, 45, and 47 and is ready for the next four segments.

Since PC-1 does not get the acknowledgment for segment number 46, it assumes that PC-2 did not receive it. It resends segment 46 in the next batch of four segments. The next time it sends the segment numbers 46, 48, 49, and 50.

This way using the sequence numbers and acknowledgment signals, TCP provides guaranteed data delivery.

Unreliable Connections

The main problem with a reliable connection is that to establish it, TCP always goes through a handshake process before data can be transferred, and then acknowledge that all data sent was received.

If you want to send only one piece of information and get a single reply back, going through a handshake process is worthless. It adds overhead and delays.

For example, a device uses a DNS query to resolve a fully qualified domain name to an IP address. The device sends the single DNS query to a DNS server and waits for the server's response.

In this process, since only two messages are generated and exchanged (the client's query and the server's response), it makes no sense to establish a reliable connection first before sending the query.

Instead of it, the device can just send its query and wait for a response. If a response doesn't come back, the device can send the query again.

Sending data without establishing a connection is called connectionless data delivery.

2. Congestion Control

What is **congestion**?

A state occurring in network layer when the message traffic is so heavy that it slows down network response time.

Effects of Congestion

- As delay increases, performance decreases.
- If delay increases, retransmission occurs, making situation worse.

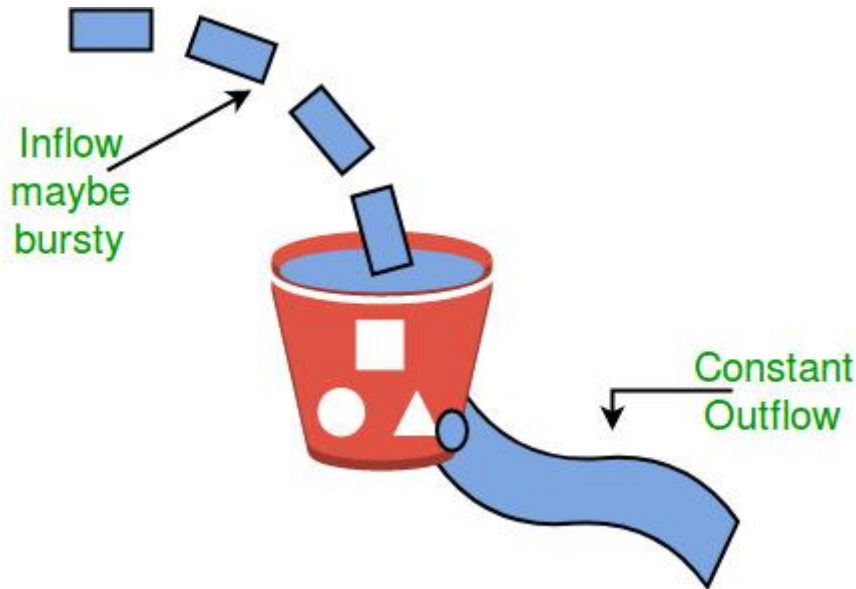
Congestion control algorithms

- Congestion Control is a mechanism that controls the entry of data packets into the network, enabling a better use of a shared network infrastructure and avoiding congestive collapse.
- Congestive-Avoidance Algorithms (CAA) are implemented at the TCP layer as the mechanism to avoid congestive collapse in a network.
- There are two congestion control algorithm which are as follows:
 - **Leaky Bucket Algorithm**
 - The leaky bucket algorithm discovers its use in the context of network traffic shaping or rate-limiting.
 - A leaky bucket execution and a token bucket execution are predominantly used for traffic shaping algorithms.
 - This algorithm is used to control the rate at which traffic is sent to the network and shape the burst traffic to a steady traffic stream.

- The disadvantages compared with the leaky-bucket algorithm are the inefficient use of available network resources.
- The large area of network resources such as bandwidth is not being used effectively.

Let us consider an example to understand

Imagine a bucket with a small hole in the bottom. No matter at what rate water enters the bucket, the outflow is at constant rate. When the bucket is full with water additional water entering spills over the sides and is lost.



Similarly, each network interface contains a leaky bucket and the following **steps** are involved in leaky bucket algorithm:

1. When host wants to send packet, packet is thrown into the bucket.
2. The bucket leaks at a constant rate, meaning the network interface transmits packets at a constant rate.
3. Bursty traffic is converted to a uniform traffic by the leaky bucket.
4. In practice the bucket is a finite queue that outputs at a finite rate.

- **Token bucket Algorithm**

- The leaky bucket algorithm has a rigid output design at an average rate independent of the bursty traffic.
- In some applications, when large bursts arrive, the output is allowed to speed up. This calls for a more flexible algorithm, preferably one that never loses information. Therefore, a token bucket algorithm finds its uses in network traffic shaping or rate-limiting.
- It is a control algorithm that indicates when traffic should be sent. This order comes based on the display of tokens in the bucket.
- The bucket contains tokens. Each of the tokens defines a packet of predetermined size. Tokens in the bucket are deleted for the ability to share a packet.
- When tokens are shown, a flow to transmit traffic appears in the display of tokens.
- No token means no flow sends its packets. Hence, a flow transfers traffic up to its peak burst rate in good tokens in the bucket.

Need of token bucket Algorithm:-

The leaky bucket algorithm enforces output pattern at the average rate, no matter how bursty the traffic is. So in order to deal with the bursty traffic we need a flexible algorithm so that the data is not lost. One such algorithm is token bucket algorithm.

Steps of this algorithm can be described as follows:

1. In regular intervals tokens are thrown into the bucket. f
2. The bucket has a maximum capacity. f
3. If there is a ready packet, a token is removed from the bucket, and the packet is sent.
4. If there is no token in the bucket, the packet cannot be sent.

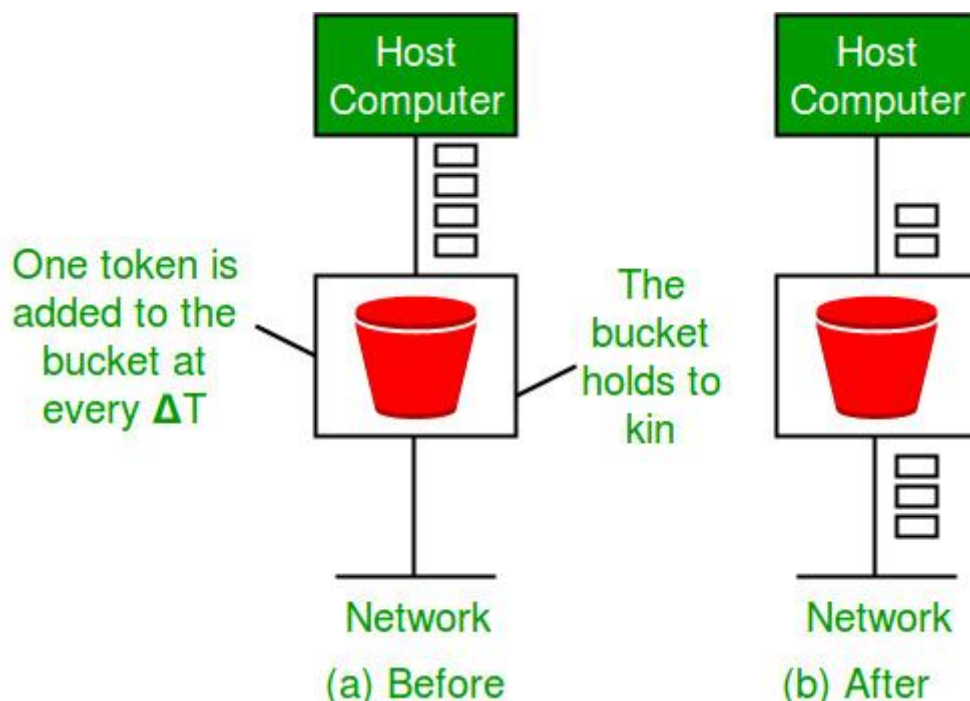
Let's understand with an example,

In figure (A) we see a bucket holding three tokens, with five packets waiting to be transmitted. For a packet to be transmitted, it must capture and destroy one token. In figure (B) We see that three of the five packets have gotten through, but the other two are stuck waiting for more tokens to be generated.

Ways in which token bucket is superior to leaky bucket: The leaky bucket algorithm controls the rate at which the packets are introduced in the network, but it is very conservative in nature. Some flexibility is introduced in the token bucket algorithm. In the token bucket, algorithm tokens are generated at each tick (up to a certain limit). For an incoming packet to be transmitted, it must capture a token and the transmission takes place at the same rate. Hence some of the bursty packets are transmitted at the same rate if tokens are available and thus introduces some amount of flexibility in the system.

Formula: $M * s = C + \rho * s$ where S – is time taken M – Maximum output rate ρ – Token arrival rate C – Capacity of the token bucket in byte

Let's understand with an example,



3. Intra-Domain Routing :

All routers inside a domain utilize an intradomain routing protocol to share routing information about the destinations that are accessible within the domain.

There are a number of intradomain routing protocols to choose from. A distance-vector protocol called RIP is used by several domains. Other domains utilize OSPF or IS-IS, which are link-state routing protocols. Finally, some domains utilize proprietary protocols like IGRP or EIGRP or use static routing.

Intradomain routing methods often have two goals. They begin by disseminating routing information that corresponds to the shortest path between two domain routers. Second, they must enable routers to recover rapidly from link and router failures.

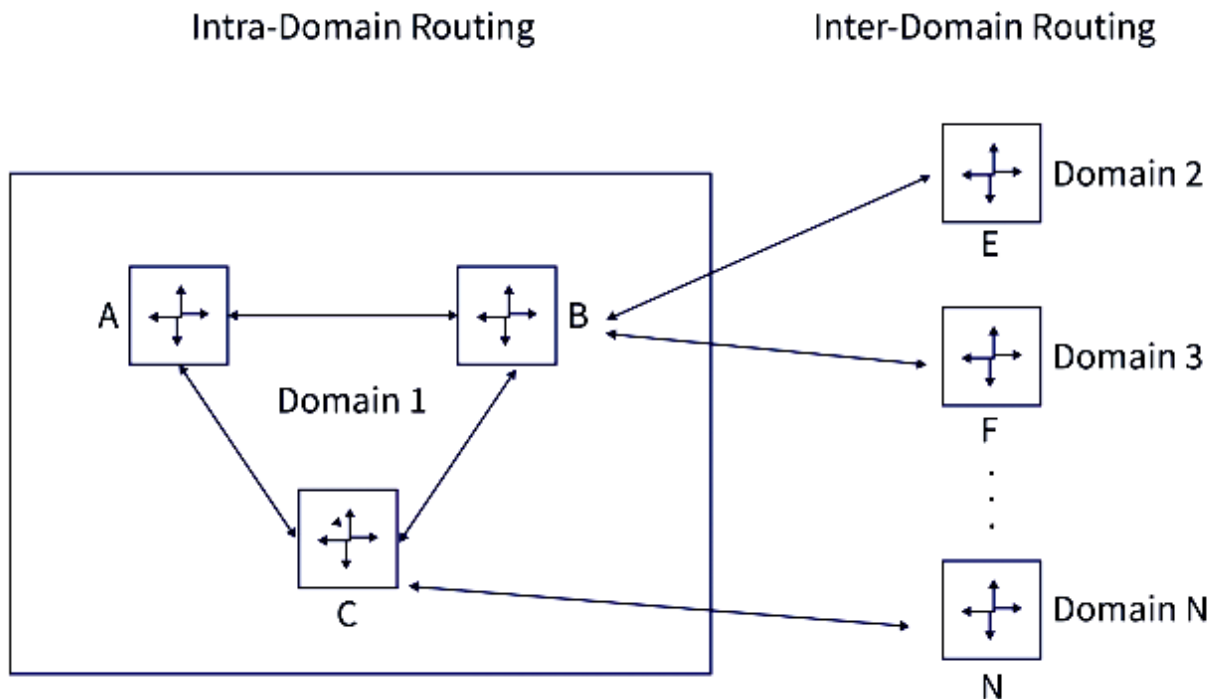
Interdomain Routing

Interdomain Routing is the protocol in which the routing algorithm works both within and between domains. Domains must be connected in some way, for hosts inside one domain to exchange data with hosts in other domains. This connection within domains is governed by the interdomain routing protocols. This is often done using the Border Gateway Protocol (BGP). It is used in Path Vector Routing using which interdomain routing is performed. In path vector routing, the routing depends on the analysis of the path from the nodes in the current domain to the node in the other domain, and not on the distance between nodes.

Intradomain Routing

Intradomain Routing is the routing protocol that operates only within a domain. In other words, intradomain routing protocols are used to route packets within a specific domain, such as within an institutional network for e-mail or web browsing. Unlike interdomain routing protocols, it doesn't communicate with other domains. There are two types of protocols used for intradomain routing:

1. **Distance Vector Routing (uses Routing Information Protocol or RIP)** In distance vector routing, each node in a domain stores information about its neighboring nodes. The information is stored in a table known as a routing table, which is maintained by each node in the domain. RIP is one of the earliest distance-vector routing protocols, and it uses hop count as a routing statistic. By placing a cap on the maximum number of hops that may be taken between a source and a destination, RIP avoids routing loops.
2. **Link State Routing (uses Open Shortest Path First or OSPF)** In link state routing, each node in a domain stores information about all the other nodes in the domain, in other words, the routing table of each node stores information about the entire topology of the domain. Since each node has all the information about the domain at its disposal, Dijkstra's algorithm is used to calculate the best routing path. This is possible due to OSPF, and this is also its advantage.



- **Distance Vector Routing**

A distance-vector routing (DVR) protocol requires that a router inform its neighbors of topology changes periodically.

- **The Distance vector algorithm is iterative, asynchronous and distributed.**
 - **Distributed:** It is distributed in that each node receives information from one or more of its directly attached neighbors, performs calculation and then distributes the result back to its neighbors.
 - **Iterative:** It is iterative in that its process continues until no more information is available to be exchanged between neighbors.
 - **Asynchronous:** It does not require that all of its nodes operate in the lock step with each other.
- The Distance vector algorithm is a dynamic algorithm.
- It is mainly used in ARPANET, and RIP.
- Each router maintains a distance table known as **Vector**.

Three Keys to understand the working of Distance Vector Routing Algorithm:

- **Knowledge about the whole network:** Each router shares its knowledge through the entire network. The Router sends its collected knowledge about the network to its neighbors.
- **Routing only to neighbors:** The router sends its knowledge about the network to only those routers which have direct links. The router sends whatever it has about the network through the ports. The information is received by the router and uses the information to update its own routing table.
- **Information sharing at regular intervals:** Within 30 seconds, the router sends the information to the neighboring routers.

Distance Vector Routing Algorithm

Let $d_x(y)$ be the cost of the least-cost path from node x to node y . The least costs are related by Bellman-Ford equation,

$$d_x(y) = \min_v \{ c(x,v) + d_v(y) \}$$

Where the $\min_v$ is the equation taken for all x neighbors. After traveling from x to v , if we consider the least-cost path from v to y , the path cost will be $c(x,v)+d_v(y)$. The least cost from x to y is the minimum of $c(x,v)+d_v(y)$ taken over all neighbors.

With the Distance Vector Routing algorithm, the node x contains the following routing information:

- For each neighbor v , the cost $c(x,v)$ is the path cost from x to directly attached neighbor, v .
- The distance vector x , i.e., $D_x = [D_x(y) : y \text{ in } N]$, containing its cost to all destinations, y , in N .
- The distance vector of each of its neighbors, i.e., $D_v = [D_v(y) : y \text{ in } N]$ for each neighbor v of x .

Distance vector routing is an asynchronous algorithm in which node x sends the copy of its distance vector to all its neighbors. When node x receives the new distance vector from one of its neighboring vector, v , it saves the distance vector of v and uses the Bellman-Ford equation to update its own distance vector. The equation is given below:

$$d_x(y) = \min_v \{ c(x,v) + d_v(y) \} \quad \text{for each node } y \text{ in } N$$

The node x has updated its own distance vector table by using the above equation and sends its updated table to all its neighbors so that they can update their own distance vectors.

Algorithm

At each node x ,

Initialization

```
for all destinations  $y$  in  $N$ :  
   $D_x(y) = c(x,y)$  // If  $y$  is not a neighbor then  $c(x,y) = \infty$   
for each neighbor  $w$   
   $D_w(y) = ?$  for all destination  $y$  in  $N$ .  
for each neighbor  $w$   
  send distance vector  $D_x = [ D_x(y) : y \text{ in } N ]$  to  $w$   
loop  
  wait(until I receive any distance vector from some neighbor  $w$ )  
  for each  $y$  in  $N$ :  
     $D_x(y) = \min_v \{ c(x,v) + D_v(y) \}$   
  If  $D_x(y)$  is changed for any destination  $y$   
  Send distance vector  $D_x = [ D_x(y) : y \text{ in } N ]$  to all neighbors  
forever
```

- **Link State Routing**

Link state routing is a technique in which each router shares the knowledge of its neighborhood with every other router in the internetwork.

The three keys to understand the Link State Routing algorithm:

- **Knowledge about the neighborhood:** Instead of sending its routing table, a router sends the information about its neighborhood only. A router broadcast its identities and cost of the directly attached links to other routers.
- **Flooding:** Each router sends the information to every other router on the internetwork except its neighbors. This process is known as Flooding. Every router that receives the packet sends the copies to all its neighbors. Finally, each and every router receives a copy of the same information.
- **Information sharing:** A router sends the information to every other router only when the change occurs in the information.

Link State Routing has two phases:

Reliable Flooding

- **Initial state:** Each node knows the cost of its neighbors.
- **Final state:** Each node knows the entire graph.

Route Calculation

Each node uses Dijkstra's algorithm on the graph to calculate the optimal routes to all nodes.

- The Link state routing algorithm is also known as Dijkstra's algorithm which is used to find the shortest path from one node to every other node in the network.
- The Dijkstra's algorithm is an iterative, and it has the property that after k^{th} iteration of the algorithm, the least cost paths are well known for k destination nodes.

Let's describe some notations:

- **$c(i, j)$:** Link cost from node i to node j . If i and j nodes are not directly linked, then $c(i, j) = \infty$.
- **$D(v)$:** It defines the cost of the path from source code to destination v that has the least cost currently.
- **$P(v)$:** It defines the previous node (neighbor of v) along with current least cost path from source to v .
- **N :** It is the total number of nodes available in the network.

Algorithm

Initialization

```
N = {A}      // A is a root node.
```

```
for all nodes v
```

```
if v adjacent to A
```

```
then  $D(v) = c(A, v)$ 
```

```
else  $D(v) = \text{infinity}$ 
```

loop

```
find w not in N such that  $D(w)$  is a minimum.
```

```
Add w to N
```

```
Update  $D(v)$  for all v adjacent to w and not in N:
```

```
 $D(v) = \min(D(v), D(w) + c(w, v))$ 
```

```
Until all nodes in N
```

4. Wireless Networks :

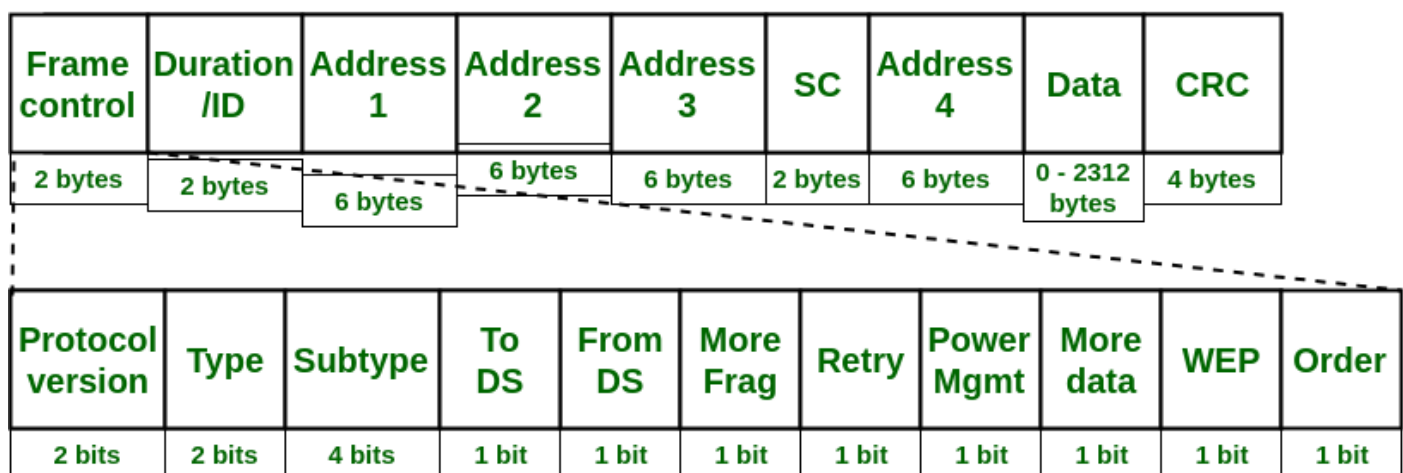
A wireless network refers to a computer network that makes use of Radio Frequency (RF) connections between nodes in the network. Wireless networks are a popular solution for homes, businesses, and telecommunications networks.

- **802.11 MAC**

Prerequisite – [Basics of Wi-fi](#) MAC layer provides functionality for several tasks like control medium access, can also offer support for roaming, authentication, and power conservation. The basic services provided by MAC are the mandatory asynchronous data service and optional time-bounded service. IEEE 802.11 defines two MAC sub-layers:-

1. **Distributed Coordination Function (DCF)** – DCF uses CSMA/CA as access method as wireless LAN can't implement CSMA/CD. It only offers asynchronous service.
2. **Point Coordination Function (PCF)** – PCP is implemented on top of DCF and mostly used for time-service transmission. It uses a centralized, contention-free polling access method. It offers both asynchronous and time-bounded service.

MAC Frame: The MAC layer frame consists of 9 fields. The following figure shows the basic structure of an IEEE 802.11 MAC data frame along with the content of the frame control field.



IEEE 802.11 MAC Frame Structure

- **Frame Control(FC)** – It is 2 bytes long field which defines type of frame and some control information. Various fields present in FC are:
 1. **Version:** It is a 2 bit long field which indicates the current protocol version which is fixed to be 0 for now.
 2. **Type:** It is a 2 bit long field which determines the function of frame i.e management(00), control(01) or data(10). The value 11 is reserved.
 3. **Subtype:** It is a 4 bit long field which indicates sub-type of the frame like 0000 for association request, 1000 for beacon.
 4. **To DS:** It is a 1 bit long field which when set indicates that destination frame is for DS(distribution system).
 5. **From DS:** It is a 1 bit long field which when set indicates frame coming from DS.
 6. **More frag (More fragments):** It is 1 bit long field which when set to 1 means frame is followed by other fragments.

7. **Retry:** It is 1-bit long field, if the current frame is a retransmission of an earlier frame, this bit is set to 1.
 8. **Power Mgmt (Power management):** It is 1-bit long field that indicates the mode of a station after successful transmission of a frame. Set to 1 the field indicates that the station goes into power-save mode. If the field is set to 0, the station stays active.
 9. **More data:** It is 1-bit long field that is used to indicate receiver that a sender has more data to send than the current frame. This can be used by an access point to indicate to a station in power-save mode that more packets are buffered or it can be used by a station to indicate to an access point after being polled that more polling is necessary as the station has more data ready to transmit.
 10. **WEP:** It is 1 bit long field which indicates that the standard security mechanism of 802.11 is applied.
 11. **Order:** It is 1 bit long field, if this bit is set to 1 the received frames must be processed in strict order.
- **Duration/ID** – It is 4 bytes long field which contains the value indicating the period of time in which the medium is occupied(in μ s).
 - **Address 1 to 4** – These are 6 bytes long fields which contain standard IEEE 802 MAC addresses (48 bit each). The meaning of each address depends on the DS bits in the frame control field.
 - **SC (Sequence control)** – It is 16 bits long field which consists of 2 sub-fields, i.e., Sequence number (12 bits) and Fragment number (4 bits). Since acknowledgement mechanism frames may be duplicated hence, a sequence number is used to filter duplicate frames.
 - **Data** – It is a variable length field which contain information specific to individual frames which is transferred transparently from a sender to the receiver(s).
 - **CRC (Cyclic redundancy check)** – It is 4 bytes long field which contains a 32 bit CRC error detection sequence to ensure error free frame.